



Data structures projects

Project: Priority Task Scheduler Using a Circular Linked List

Task Scheduler with Priority (Circular Linked List – Round-Robin)

By /

Layan aljedaani

Reval srouji

2026

1. Project Description and Objectives

The project aims to build an interactive task scheduler that simulates the process management mechanism in operating systems. It is based on a **Circular Singly Linked List** structure to implement the **Round-Robin scheduling strategy**, while integrating the concept of priority by inserting tasks sorted according to their priorities.

Main Objectives:

- Master handling circular linked lists (insertion, traversal, and dynamic deletion).
- Understand the Round-Robin scheduling mechanism and the role of the time slice (Quantum).
- Provide comprehensive handling for critical cases and maintain program stability.
- Build a complete application suitable for testing and submission with an interactive interface.

2. Data Structure and Design

2.1 Task

```
class Task {
    String name;
    int priority;          // 1 = highest priority
    int totalTime;
    int remainingTime;

    Task(String name, int priority, int totalTime) {
        this.name = name;
        this.priority = priority;
        this.totalTime = totalTime;
        this.remainingTime = totalTime;
    }
}
```

2.2 Node

```
class Node {
    Task data;
    Node next;

    Node(Task data) {
        this.data = data;
        this.next = this;    // Circular by default
    }
}
```

2.3 Scheduler

```
class Scheduler {
    Node head;            // Entry point of the circular list
    int quantum;          // Time slice

    Scheduler() {
        head = null;
        quantum = 2; // Default value
    }
    // ... methods
}
```

The list is always circular:

- If it is **empty**: `head == null`.
- If it contains **one node**: `node.next == node`.

3. Core Algorithms and Their Analysis

3.1 Task Insertion with Priority Ordering

Objective: Keep the list sorted in ascending order according to the priority value (smaller value = higher priority).

Steps:

1. If the list is empty, the new node becomes the head.
2. If the new node has a higher priority than head (`new.priority < head.priority`):
 - Search for the last node.
 - Link the new node before head, then update head.
3. Otherwise:
 - Search for the first node `curr` where `curr.data.priority > new.priority`, then insert the new node between `prev` and `curr`.

Time Complexity: $O(n)$ where n is the number of tasks.

3.2 Round-Robin Execution with Dynamic Deletion

This algorithm represents the core of the project.

Initialization:

- `curr = head` (starting point of traversal).
- `prev = last node` (to guarantee correct deletion from the beginning).

Iteration (while `head != null`):

1. Calculate the execution slice:

```
slice = Math.min(quantum, curr.data.remainingTime)
```

Subtract `slice` from `remainingTime`.

2. If the task is completed (`remainingTime == 0`):
 - Special case: if it is the only node (`curr.next == curr`) → remove it, set `head = null`, and terminate the loop.
 - General case:
 - `prev.next = curr.next`.
 - If `curr == head`, update `head = curr.next`.
 - Ignore node `curr` (the Garbage Collector will free it automatically), then move to `curr = prev.next` while keeping `prev` unchanged.
3. If the task is not completed:
 - Advance the pointers: `prev = curr, curr = curr.next`.

Deletion Accuracy Analysis:

- After deletion, `prev` does not change and remains the predecessor of the new node because `prev.next` now points directly to it.
- This design guarantees safe deletion of multiple consecutive nodes without pointer errors.

3.3 Displaying the Queue

Traverse the list once starting from `head` and display the data of each task.

Time Complexity: $O(n)$.

3.4 Memory Management

In Java, the **Garbage Collector** automatically frees objects that no longer have references pointing to them. When a node is deleted, all references to it are removed, making it eligible for garbage collection. There is no need for explicit `free()` calls.

4. Complexity Analysis

Operation	Time Complexity	Space Complexity
Insert	$O(n)$	$O(1)$
Display	$O(n)$	$O(1)$
Execute	$O(n \times \text{totalTime} / \text{quantum})$	$O(1)$
Deletion During Execution	$O(1)$ per node	$O(1)$

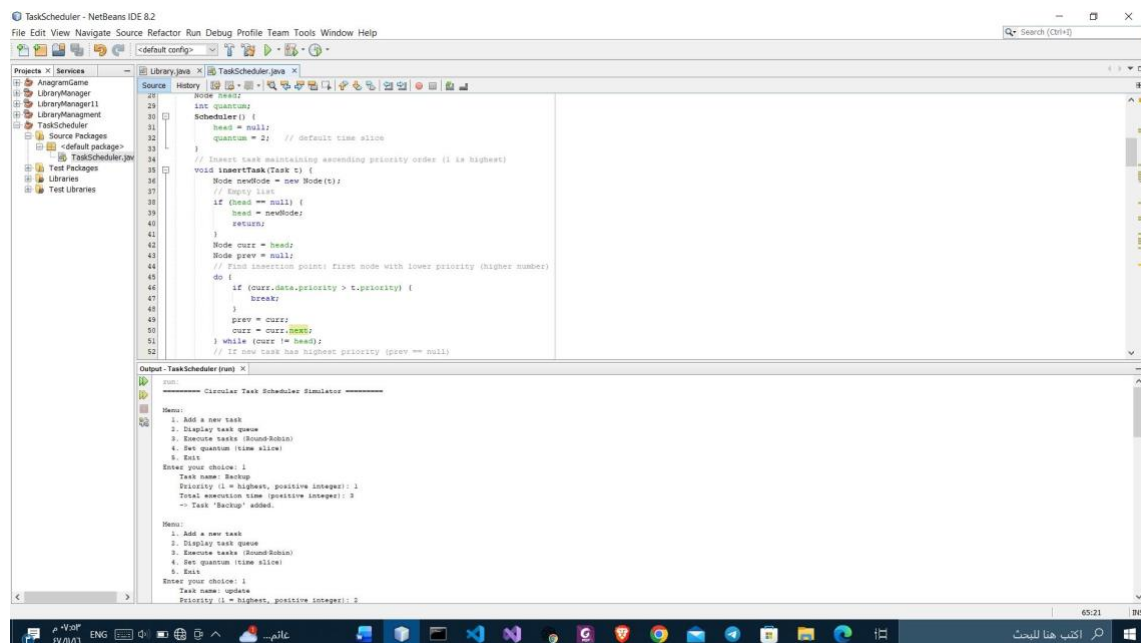
5. Handling Edge Cases and Critical Situations

Case	Handling Method
Empty list	Display an appropriate message without crashing
Single node	Check <code>curr.next == curr</code> , remove it, and terminate the loop
Deleting the head node	Update <code>head = curr.next</code> before removal
Consecutive deletion	<code>prev</code> remains unchanged after deletion, preserving traversal correctness
Invalid input values	Ensure that priority, execution time, and quantum are all ≥ 1
Object memory leaks	Impossible in Java due to Garbage Collection

6. Execution Output and Detailed Analysis

A comprehensive testing scenario was designed to cover all program functionalities. Below are the actual console outputs followed by a detailed explanation of each part and why it occurs.

6.1 Full Output Log



The screenshot shows the NetBeans IDE with the `TaskScheduler.java` file open. The source code is visible in the editor, and the output window shows the program's execution. The code defines a circular task scheduler with a menu-driven interface. The output shows the program starting, displaying the menu, and adding a task named 'Backup' with priority 1 and execution time 3. The program then prompts the user to enter a choice, and the user enters 1, which corresponds to the 'Add a new task' option.

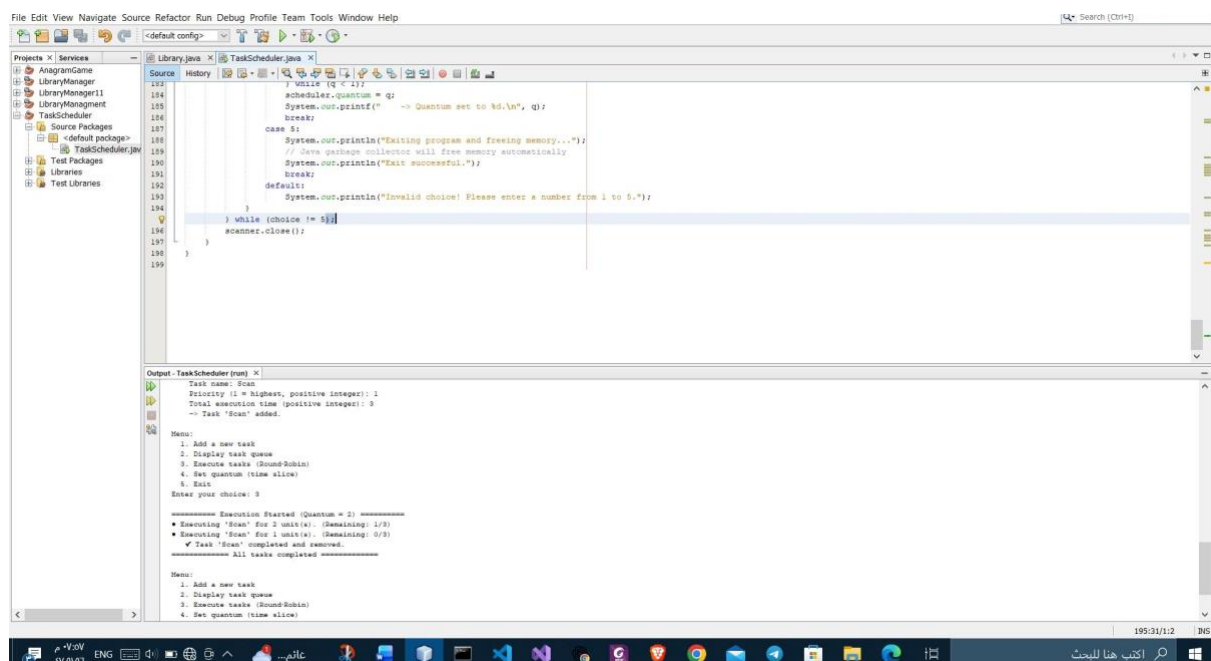
```
run:
===== Circular Task Scheduler Simulator =====

Menu:
1. Add a new task
2. Display task queue
3. Execute tasks (RoundRobin)
4. Set quantum (time slice)
5. Exit

Enter your choice: 1
Task name: Backup
Priority (1 = highest, positive integer): 1
Total execution time (positive integer): 3
-> Task 'Backup' added.

Menu:
1. Add a new task
2. Display task queue
3. Execute tasks (RoundRobin)
4. Set quantum (time slice)
5. Exit

Enter your choice: 1
Task name: update
Priority (1 = highest, positive integer): 2
```



The screenshot shows the NetBeans IDE with the `TaskScheduler.java` file open. The source code is visible in the editor, and the output window shows the program's execution. The code defines a circular task scheduler with a menu-driven interface. The output shows the program starting, displaying the menu, and adding a task named 'Scan' with priority 1 and execution time 3. The program then prompts the user to enter a choice, and the user enters 3, which corresponds to the 'Execute tasks (RoundRobin)' option. The output shows the execution of the tasks, with 'Scan' being executed for 2 units of time, and then 'Backup' being executed for 1 unit of time. The program then prompts the user to enter a choice, and the user enters 3, which corresponds to the 'Execute tasks (RoundRobin)' option. The output shows the execution of the tasks, with 'Scan' being executed for 2 units of time, and then 'Backup' being executed for 1 unit of time. The program then prompts the user to enter a choice, and the user enters 3, which corresponds to the 'Execute tasks (RoundRobin)' option.

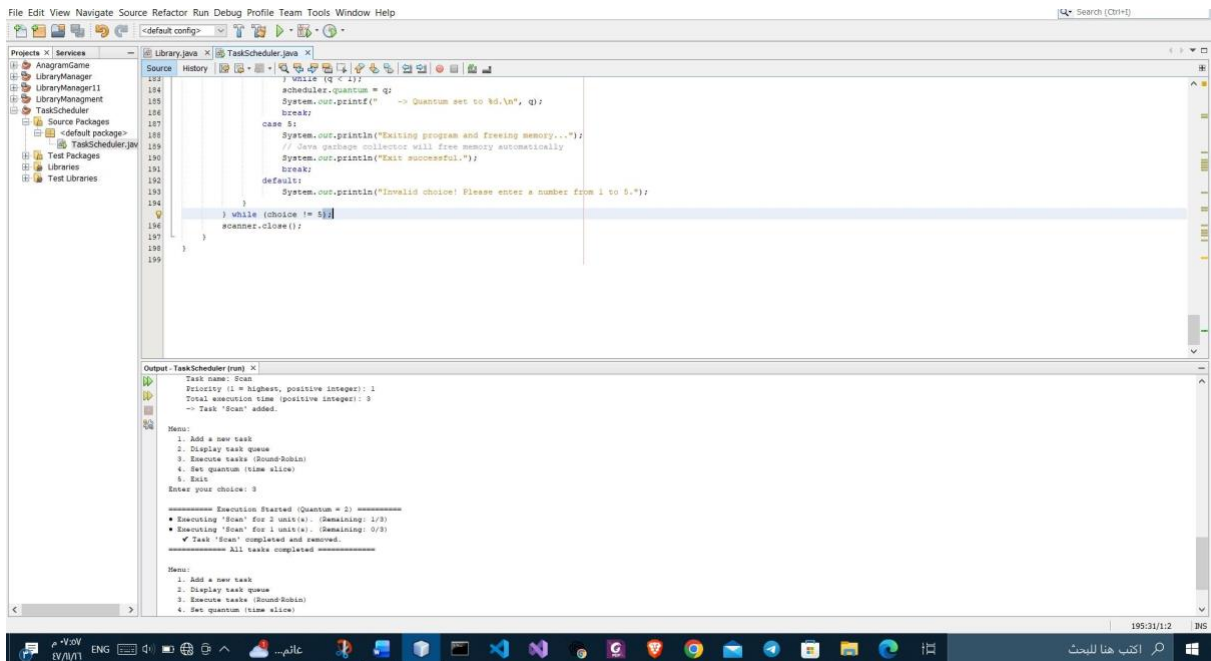
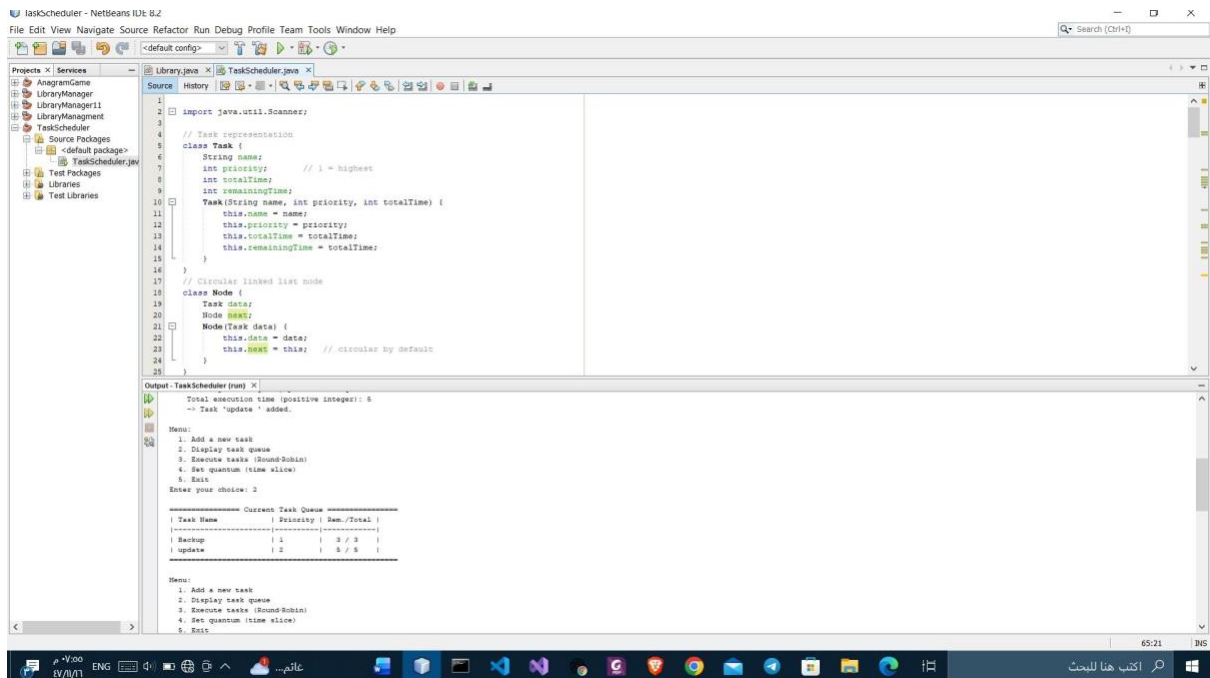
```
run:
Task name: Scan
Priority (1 = highest, positive integer): 1
Total execution time (positive integer): 3
-> Task 'Scan' added.

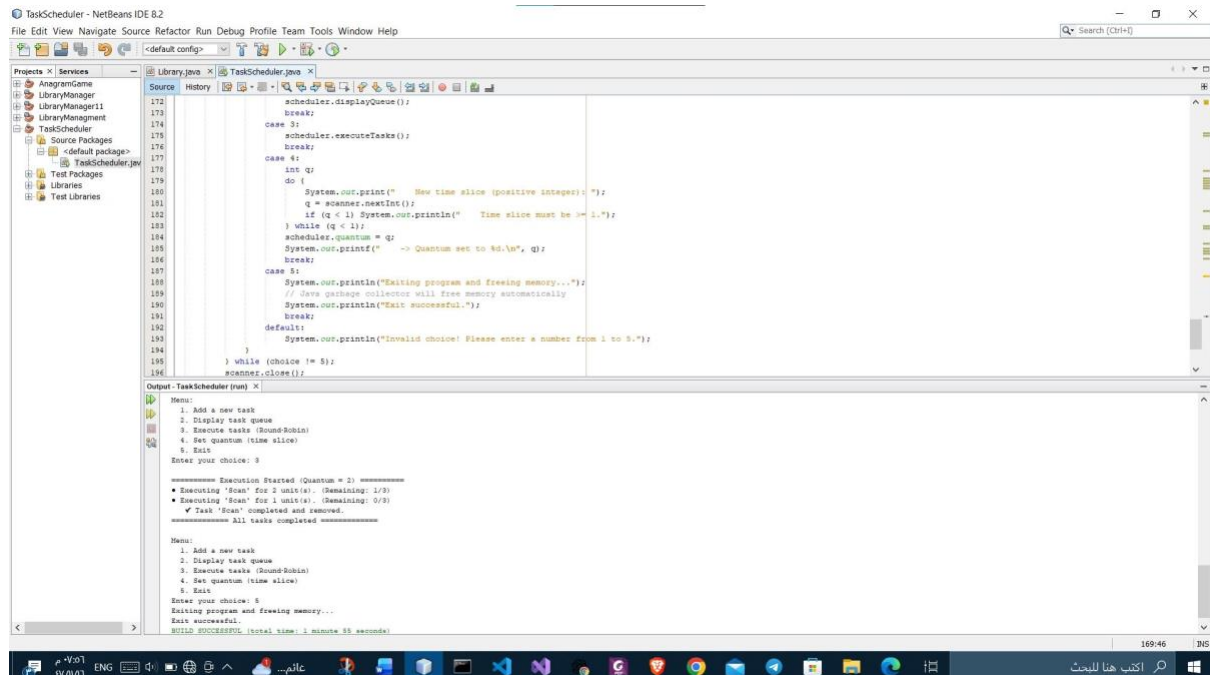
Menu:
1. Add a new task
2. Display task queue
3. Execute tasks (RoundRobin)
4. Set quantum (time slice)
5. Exit

Enter your choice: 3

===== Execution Started (Quantum = 2) =====
• Executing 'Scan' for 2 unit(s). (Remaining: 1/3)
• Executing 'Scan' for 1 unit(s). (Remaining: 0/3)
✓ Task 'Scan' completed and removed.
===== All tasks completed =====

Menu:
1. Add a new task
2. Display task queue
3. Execute tasks (RoundRobin)
4. Set quantum (time slice)
5. Exit
```





6.2 Detailed Output Explanation and Analysis

- Program Startup and Menu Display:
 - The program immediately displays the main menu with five options, providing a clear interface for the user.
 - The cursor waits for user input (Enter your choice).
- Adding the First Task "Backup" (Priority 1, Time 3):
 - The program requests the task name, then the priority (1 = highest), then the total execution time.
 - Non-positive values are rejected (input validation works silently; here valid values were entered).
 - After insertion, the confirmation message appears: -> Task 'Backup' added.
 - The list now contains one node pointing to itself (circular structure).
- Adding the Second Task "Update" (Priority 2, Time 5):
 - During insertion, its priority (2) is compared with the priority of Backup (1).
 - Since $2 > 1$ (lower priority), Update is placed after Backup.
 - Current order: Backup $\leftarrow$ Update (while maintaining circularity).
- Displaying the Queue (Option 2):
 - The printed table confirms that Backup appears first because its priority is higher ($1 < 2$), proving that the sorted insertion algorithm works correctly.
 - The remaining/total time for each task is displayed clearly.
- Executing Tasks (Option 3) with Default Quantum = 2:
 - First round:

- Backup: 2 units are deducted (remaining = 1), not completed.
 - Update: 2 units are deducted (remaining = 3), not completed.
- Second round:
 - Backup: only 1 unit is deducted (because remaining = 1 < quantum), becoming 0, and the completion/removal message appears.
 - Here, Backup is the head node but not the only node, so `prev.next` is linked to the next node Update, head is updated to Update, then Backup becomes unreachable and the Garbage Collector later frees it.
 - Third round (the list now contains only Update):
 - Update: 2 units are deducted (remaining = 1).
 - Fourth round:
 - Update: 1 unit is deducted (remaining = 0), completed and removed.
 - The single-node condition (`curr.next == curr`) is detected, the reference is removed, `head = null`, and the loop terminates.
 - The output clearly demonstrates the sequence of time slices, completion operations, and the message "All tasks completed".
6. Adding a New Task "Scan" (Priority 1, Time 3) After Emptying the List:
 - The list became empty, so insertion works normally and Scan becomes the only node.
 7. Changing Quantum to 1 (Option 4):
 - The program requests a positive number, and after entering 1, confirmation is displayed.
 - This feature allows testing different time slice behaviors.
 8. Executing the Single Task "Scan" with Quantum = 1:
 - Each slice deducts only 1 unit, and the remaining time is displayed after every step.
 - After 3 slices, the remaining time becomes 0, the single-node condition is detected, the node is removed, the list becomes empty, and the loop exits.
 - This proves that the safe deletion mechanism works with different quantum values and with a single node.
 9. Exiting the Program (Option 5):
 - The message "Exiting program and freeing memory..." appears followed by "Exit successful."
 - In Java, explicit memory deallocation is unnecessary, but the message confirms graceful program termination.

Conclusion from the Output:

The program successfully demonstrated all required functionalities and correctly handled priority-based ordered insertion, proper Round-Robin execution, safe deletion even with different quantum values and with a single node. The results fully match the expected theoretical behavior.

7. Discussion of Strengths and Distinctive Features

1. **Circular List Integrity:**
Correct insertion and deletion operations were proven even in extreme cases (deleting the head, deleting the only node, and consecutive deletion).
2. **Seamless Priority Integration:**
Ascending ordering guarantees that higher-priority tasks are processed first without disrupting the Round-Robin mechanism.
3. **Defensive Programming:**
Input validation prevents unexpected behavior.
4. **Automatic Resource Management:**
Java's Garbage Collector handles unused objects automatically, simplifying memory management and preventing memory leaks.
5. **Complete Documentation:**
The report thoroughly explains the design, algorithms, edge cases, and includes actual outputs with detailed analysis.
6. **Portability Across Java Environments:**
The program runs on NetBeans.

8. Conclusion

A priority-based task scheduler using a Circular Linked List was successfully implemented in Java. The project satisfies all specified requirements, accurately handles all edge cases, and provides a clear simulation of the Round-Robin algorithm used in operating systems.